

Mobility Lens

Assignment Report

Aayush Kumar, U73761402

Package: `io.github.aayushhks.mobilitylens`

Emulator: Medium Phone (AVD), Android 16, API 36

SDK configuration

Setting	Value
<code>minSdk</code>	27 (Android 8.1)
<code>targetSdk</code>	36 (Android 16)
<code>compileSdk</code>	36 (Android 16)

`compileSdk` is the SDK version my code is compiled against. It decides which platform classes and methods I am allowed to call. It does not decide which phones can install the app. In my build file it is written with the newer block syntax, `compileSdk { version = release(36) }`, which means the same thing as `compileSdk = 36`.

`targetSdk` tells the system the newest Android version I have tested the app on. Because it is set to 36, the app accepts all behaviour changes up to Android 16, and the system does not apply older compatibility fixes for me.

`minSdk` is the lowest version the app supports. A phone running anything older than API 27 cannot install it.

I picked `minSdk = 27` because it is a good middle point. It still covers almost all Android phones that people actually use today, and it gives me adaptive launcher icons, notification channels and the `java.time` classes without any extra setup. If I went down to API 21 I would only gain a small number of very old devices, and I would have to do extra work to keep things compatible. If I went higher I would block usable phones for no reason, because nothing in this app needs a new API. The main compatibility point is that Compose and Material 3 run on every supported device through the AndroidX libraries. If I later added something that only exists on API 33 and above, I would have to wrap it in a version check.

Project structure

MainActivity.kt sits in `app/src/main/java/io/github/aayushhks/mobilitylens/`. It is the only activity and the entry point of the app. It calls `setContent` so that Compose draws the whole screen, and it hosts my `MobilityLensScreen` composable.

AndroidManifest.xml sits in `app/src/main/`. It declares the app, its icon, label and theme, and it marks `MainActivity` as the launcher activity using a `MAIN` and `LAUNCHER` intent filter. Permissions would also go here. My app declares none, because it does not read any data from the device.

The app level Gradle file is `app/build.gradle.kts`. It sets the namespace and `applicationId`, the three SDK values, the Java and Kotlin targets, turns on the Compose build feature, and lists all the dependencies for this module.

The version catalog is `gradle/libs.versions.toml`. It keeps every library and plugin version

in one place. Other files refer to them by name, such as `libs.androidx.compose.material3`, so the versions stay the same everywhere.

`strings.xml` sits in `app/src/main/res/values/`. It holds all the text the user sees: the title, the introduction, the button labels, the validation messages and the text for all six dimensions. Keeping text here separates it from the code, and adding another language later just means adding a `values-xx/` folder.

Rotation observation

I moved to dimension 4, typed a feature name into the text field, and then rotated the emulator. After the rotation the app went back to dimension 1, the text field was empty, and the message under the button was gone. Nothing that I had done survived.

This happens because all three values are stored with `remember`. That only keeps a value alive while the same composition is running. Rotating the phone is a configuration change, so Android destroys the activity and creates it again, and Compose builds the screen fresh with the starting values.

An app needs to keep this information because the user did the work, not the app. Losing typed text because someone turned their phone feels like a bug to the user. The usual fix is `rememberSaveable` or a `ViewModel`, since both of those survive a configuration change.

One mobile design decision

I show one dimension at a time with Previous and Next buttons instead of putting all six on a single long page. On a desktop there is enough width and enough time in one sitting to show everything side by side and compare. On a phone people usually open an app for a few seconds while doing something else, so one idea per screen is easier to read. The “4 of 6” label and the progress bar tell the user where they are, and the two wide buttons sit near the bottom of the screen where a thumb can reach them.

Collaboration

I did not collaborate with any classmates on this assignment.

Generative-AI disclosure

Tool used: ChatGPT by OpenAI.

I used ChatGPT to help me understand Kotlin syntax, Jetpack Compose, and library usage alongside the official documentation. The prompts below summarize my questions rather than give their exact wording.

1. Navigation

Prompt summary: “How can I manage Previous and Next navigation in Compose?”

Suggestion and what I used: ChatGPT suggested using a state-based index and disabling navigation at the list ends. I applied this to my six dimensions.

Verification: I tested the buttons in the emulator and checked that the displayed dimension changed correctly.

2. Rotation and state

Prompt: “Why does my text disappear when I rotate the emulator?”

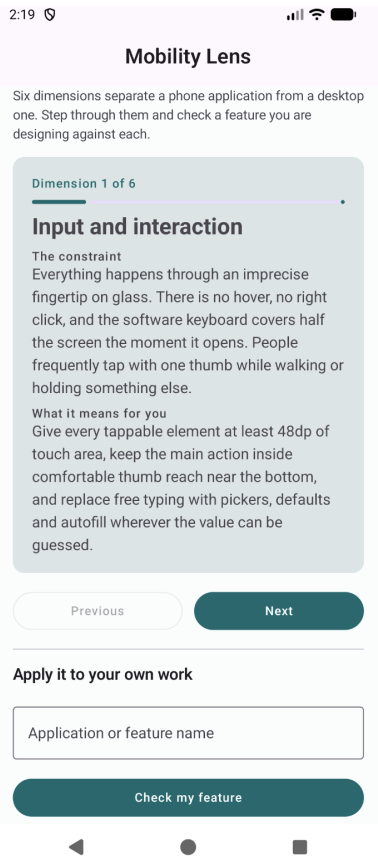
Suggestion and what I used: ChatGPT explained that `remember` does not preserve values when the activity is recreated and suggested `rememberSaveable`. I kept `remember` because fixing rotation was not required for this assignment.

Verification: I entered text on dimension 4 and rotated the emulator. The app returned to dimension 1 and cleared the text, as recorded in my report.

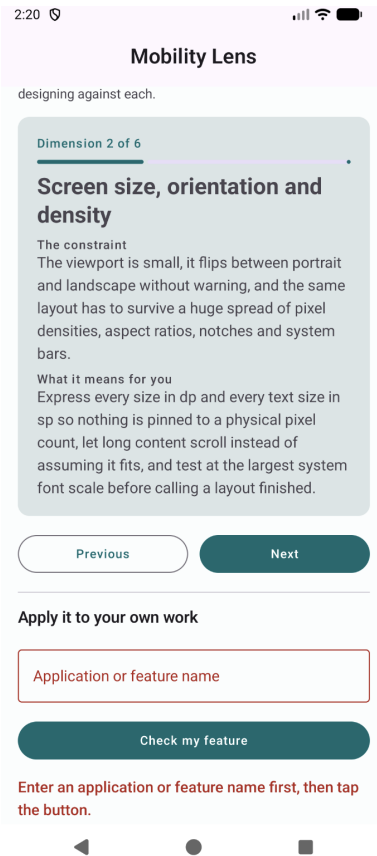
Dimension content

ChatGPT also helped me understand the six mobility dimensions and suggested examples of mobile constraints and developer recommendations.

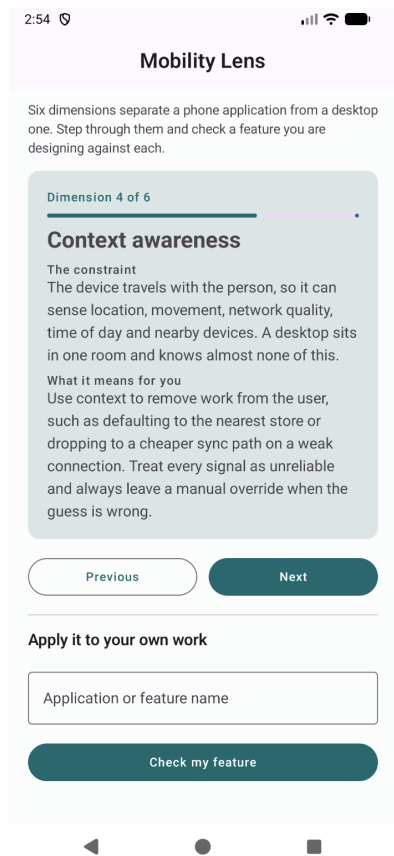
Screenshots (next page)



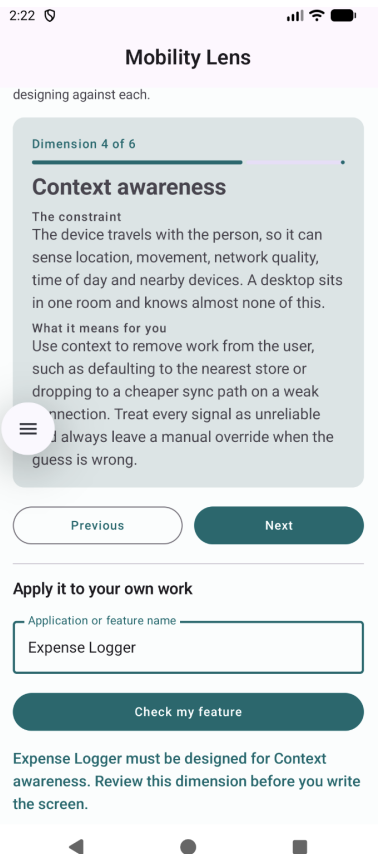
(a) First dimension on launch.



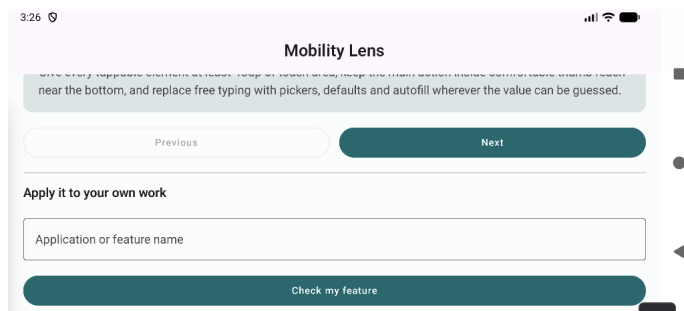
(b) Blank input rejected.



(c) A different dimension after tapping Next.



(d) Valid input accepted.



(e) After rotating the emulator the app is back on dimension 1 with an empty field.